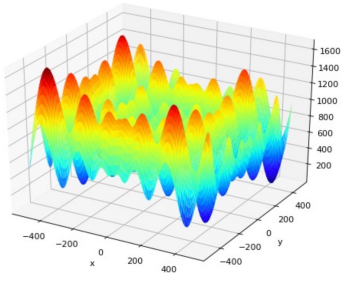
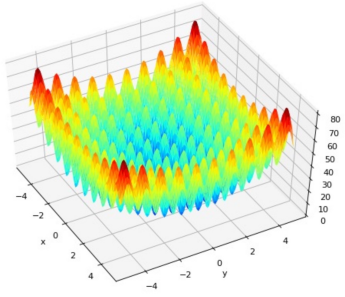
Natural Computing

LMU Munich
summer term 2025

Thomas Gabor

The Solution Landscape Metaphor





source: deap.readthedocs.io/en/master/api/benchmarks.html

Algorithm 1 (single-sample random search)**.** Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. The process $\mathcal{D}$ continues via (single-sample) random search if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \underset{x \in \{x_t, x'_t\}}{\arg\min} \ \tau(x)$$

where $x'_t \sim \mathcal{X}$ is drawn at random for each call to e.

Algorithm 2 (stochastic hill climbing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $\mathit{neighbors} : \mathcal{X} \to \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. The process $\mathcal{D}$ continues via stochastic hill climbing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \operatorname*{arg\,min}_{x \in \{x_t, x'_t\}} \tau(x)$$

where $x'_t \sim \mathit{neighbors}(x_t)$ is drawn at random for each call to e.

Algorithm 3 (simulated annealing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $\mathit{neighbors} : \mathcal{X} \to \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. Let $\kappa : \mathbb{N} \to \mathbb{R}$ be a temperature schedule, i.e., a function that returns a temperature value for each time step. Let $A : \mathcal{T} \times \mathcal{T} \times \mathbb{R} \to \mathbb{P}$ with $\mathbb{P} = [0; 1] \subset \mathbb{R}$ be a function that returns an acceptance probability given two target values and a temperature. Commonly, we use

$$A(Q, Q', K) = \mathrm{e}^{\frac{-(Q'-Q)}{K}}$$

for $\mathcal{T} \subseteq \mathbb{R}$ and Euler's number e. The process $\mathcal{D}$ continues via simulated annealing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} x'_t & \text{if } \tau(x'_t) \leq \tau(x_t) \text{ or } r \leq A(\tau(x_t), \tau(x'_t), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where $x'_t \sim \mathit{neighbors}(x_t)$ and $r \sim \mathbb{P}$ are drawn at random for each call to e.

Algorithm 4 (gradient descent)**.** Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $\mathcal{T}$ be continuous ($\mathcal{T} = \mathbb{R}$, e.g.) and let $\tau' : \mathcal{X} \to \mathcal{T}$ be the first derivative of τ. The process $\mathcal{D}$ continues via gradient descent (with update rate $\alpha \in \mathbb{R}^+$) if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha \cdot \tau'(x_t).$$

The learning rate α can also be given as a function, usually $\alpha : \mathbb{N} \to \mathbb{R}$ so that $e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha(t) \cdot \tau'(x_t)$.
If τ is stochastic, this process is called stochastic gradient descent (SGD).

Algorithm 5 (simulated annealing with local gradient descent)**.** For $n \in \mathbb{N}$, $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{D}}, \langle x_u \rangle_{0 \leq u \leq t})$ is an optimization process that continues via simulated annealing with n steps of local gradient descent if $e_{\mathcal{D}}$ has the form

$$e_{\mathcal{D}}(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} {}^*\mathcal{E}(x'_t) & \text{if } \tau({}^*\mathcal{E}(x'_t)) \leq \tau(x_t) \\ & \quad \text{or } r \leq A\big(\tau(x_t), \tau({}^*\mathcal{E}(x'_t)), \kappa(t)\big), \\ x_t & \text{otherwise,} \end{cases}$$

where x'_t, r, A, κ are given as in Alg. 3 and where $\mathcal{E}(x'_t) = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{E}}, \langle x'_{t+v} \rangle_{0 \leq v \leq n})$ is an optimization process that continues via gradient descent (cf. Alg. 4) so that

$${}^*\mathcal{E}(x'_t) = \operatorname*{arg\,min}_{\substack{x'_{t+v}, \\ 0 \leq v \leq n}} \tau(x'_{t+v})$$

is the best solution found by $\mathcal{E}$ when starting at x'_t.

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
  $m_0 \leftarrow 0$ (Initialize 1st moment vector)
  $v_0 \leftarrow 0$ (Initialize 2nd moment vector)
  $t \leftarrow 0$ (Initialize timestep)
  while θ_t not converged **do**
    $t \leftarrow t + 1$
    $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
    $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
    $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
    $\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
    $\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
    $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
  end while
  return θ_t (Resulting parameters)

How do these algorithms work?

Local vs. Global Information

Smoothness

Deception

The Exploration/Exploitation Dilemma

Which algorithm do we choose?

No Free Lunch Theorem

Theorem 3 (no free lunch [1, 3])**.** As measured by sample efficiency, i.e., the achieved minimal value of τ per evaluations of $\tau(x)$ for some new $x \in \mathcal{X}$ for finite $\mathcal{X}$, all optimization algorithms perform the same when averaged over all possible target functions τ. So, for any search/optimization algorithm, any elevated performance over one class of problems is exactly paid for in performance over another class.